

Wordle 大作业报告

一、 程序结构和说明

整个程序的主体在 main 函数中，大致包含以下几个部分：

- 1、命令行参数处理 command line arguments execution。
- 2、游戏部分 game。
- 3、其余部分（交互模式输出）。

其中命令行参数处理部分按顺序包括以下部分：

- 1、命令行参数处理变量声明。
- 2、config 参数检查和 config 初始化。
- 3、其他命令行参数读入。
- 4、命令行参数矛盾处理。
- 5、各个命令行参数初始化和检查（如 word 和指定 list 的检查）。

游戏部分按顺序按顺序包括以下部分：

- 1、游戏状态初始化（若命令行有 state 参数则已经在上一部分由 json 初始化）。
- 2、游戏循环。
- 3、存档模式下的游戏存档。

其中游戏循环按顺序包括以下部分：

- 1、键盘（字母表状态）的初始化。
- 2、答案输入/生成（包括指定答案模式、随机模式）。
- 3、猜测循环。
- 4、打印游戏状态。
- 5、stats 模式下的打印游戏数据。
- 6、不指定答案模式下的询问是否下一局。

其中猜测循环按顺序包括以下部分：

- 1、wordle solver 提示模式下的生成提示。
- 2、猜测输入。
- 3、困难模式下的检查。
- 4、普通模式下的猜测与答案匹配。
- 5、打印猜测状态。
- 6、判断输赢。

整体结构类似下图：

- 命令行参数处理command line arguments execution
 1. 命令行参数处理变量声明
 2. config参数检查和config初始化
 3. 其他命令行参数读入
 4. 命令行参数矛盾处理
 5. 各个命令行参数初始化和检查（如word和指定list的检查）
- 游戏部分game
 1. 游戏状态初始化（若命令行有state参数则已经在上一部分由json初始化）
 2. 游戏循环
 1. 键盘（字母表状态）的初始化
 2. 答案输入/生成（包括指定答案模式、随机模式）
 3. 猜测循环
 1. wordle solver提示模式下的生成提示
 2. 猜测输入
 3. 困难模式下的检查
 4. 普通模式下的猜测与答案匹配
 5. 打印猜测状态
 6. 判断输赢
 4. 打印游戏状态
 5. stats模式下的打印游戏数据
 6. 不指定答案模式下的询问是否下一局
 3. 存档模式下的游戏存档
- 其余部分（交互模式输出）

说明：

- 1、基础版本写在 src/main.rs 中，提高版本写在 src/main+tui+hint.rs 中。
- 2、基础版本不含 tui 及 hint 功能。
- 3、基础版本若要手动测试加命令行参数通过 cargo run --bin wordle -- other_arg 运行，提高版本加命令行参数通过 cargo run --bin extra_version -- other_arg 运行。
- 4、基础版本的自动化测试通过 cargo test [--release] -- --test-threads=1 运行。
- 5、基础版本中用 better comments 插件高亮/为一些注释添加了颜色，为更好划分不同部分的代码块。
- 6、提高版本加入到 bin，在 toml 文件中的信息为：[[bin]] name = "extra_version" path = "src/main+tui+hint.rs"。

二、 游戏的主要功能、截图

游戏功能：实现了 wordle 游戏，基础版本提供了基础要求中交互/测试模式，指定答案/随机/困难/显示数据/指定词库/保存文件/配置文件模式，提高版本提供了上述模式和提高要求中的 tui 界面+wordle 求解所有可能的答案、单步提示模式。

游戏运行截图：

以交互模式、随机模式（随机输入了种子和日期）、显示数据模式为例，其余模式的界面于此类似：

基础版本：

- 1、进入游戏

```
root@LAPTOP-T6LKLHA:/mnt/d/university_study/first_year_summer/rust/wordle/wordle-wanzh22# cargo run --bin wordle -- -r
-s 1241513 -d 235 -t
```

- 2、输入姓名，开始游戏

```
Your name: wanzh
Welcome to wordle, wanzh!
```

```
Please Enter Your Guess:|
```

3、猜测后的反馈

```
Please Enter Your Guess:crane

Your Guess:
crane

Alphabet State:
abcdefghijklmopqrstuvwxyz

Please Enter Your Guess:|
```

```
Please Enter Your Guess:slate

Your Guess:
crane
slate

Alphabet State:
abcdefghijklmopqrstuvwxyz

Please Enter Your Guess:|
```

```
Please Enter Your Guess:drink

Your Guess:
crane
slate
audio
drink

Alphabet State:
abcdefghijklmnopqrstuvwxyz

Please Enter Your Guess:|
```

4、一局结束后的反馈

```
Your Guess:
crane
slate
audio
drink
grind

Alphabet State:
abcdefghijklmnopqrstuvwxyz

The Game 1 Result:
CORRECT! You Guessed 5 Times.
```

```
Win count:1, Lose count:0, Average Try count: 5.00
```

```
The Top 5 Words You Guessed Most Frequently:
AUDIO 1 CRANE 1 DRINK 1 GRIND 1 SLATE 1
```

```
Start A New Game or Not? Y for Yes, N for Not
```

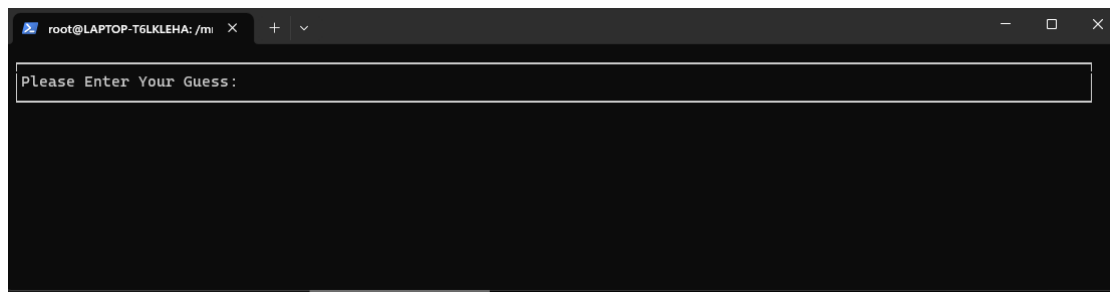
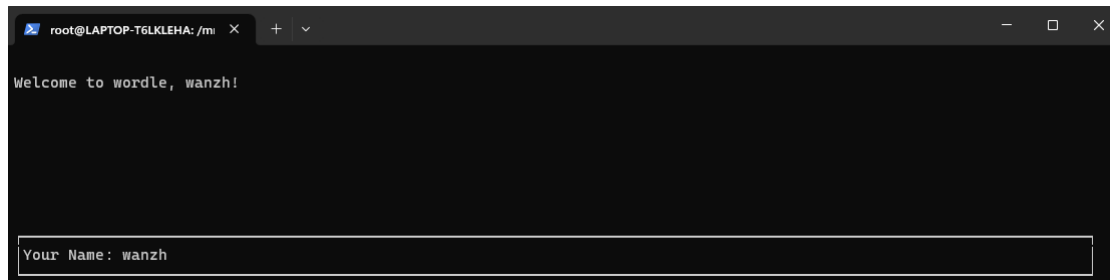
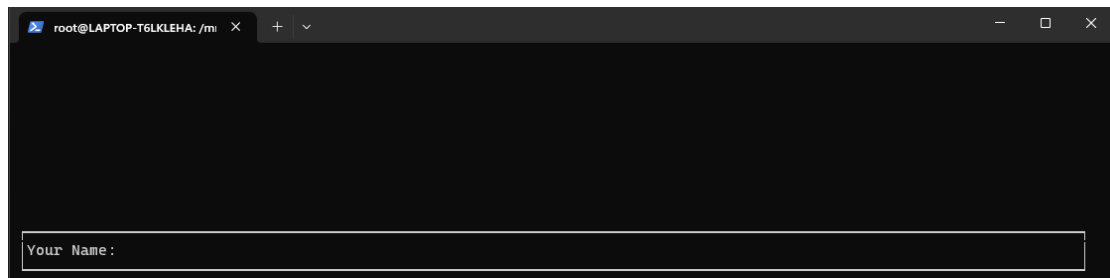
提高版本：

1、进入游戏

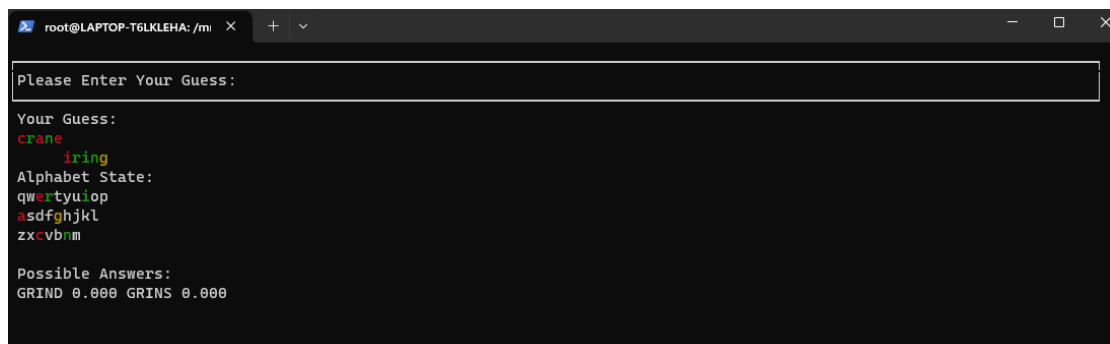
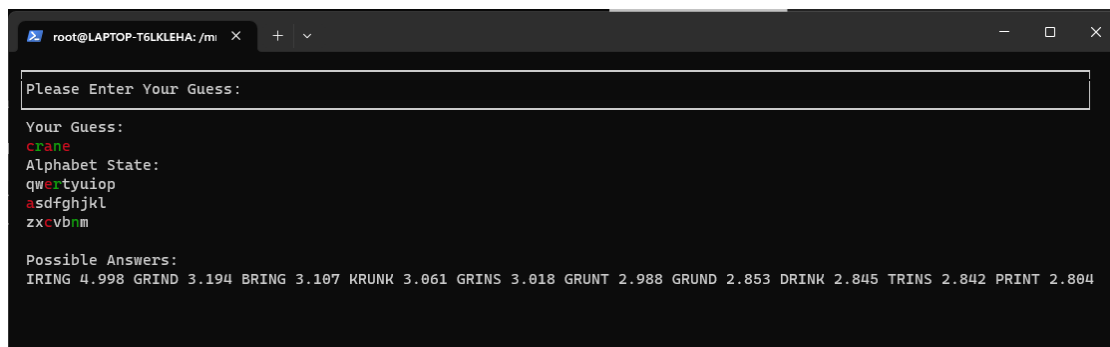
```
root@LAPTOP-T6LKLEHA: /mi  x  +  v
安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS D:\university_study\first_year_summer\rust\wordle\wordle-wanzh22> wsl
root@LAPTOP-T6LKLEHA:/mnt/d/university_study/first_year_summer/rust/wordle/wordle-wanzh22# cargo run --bin extra_version
-- -r -s 1241513 -d 235 -t -h|
```

2、输入姓名，开始游戏



3、猜测后的反馈



4、一局结束后的反馈

```
root@LAPTOP-T6LKEHA: /m  X + v

Your Guess:
crane
  iring
    grind
Alphabet State:
qwertyuiop
asdfghjkl
zxcvbnm

The Game 1 Result:
CORRECT! You Guessed 3 Times.

root@LAPTOP-T6LKEHA: /m  X + v

Your Guess:
crane
  iring
    grind
Alphabet State:
qwertyuiop
asdfghjkl
zxcvbnm

The Game 1 Result:
CORRECT! You Guessed 3 Times.
Win count:1, Lose count:0, Average Try count: 3.00

The Top 5 Words You Guessed Most Frequently:CRANE 1 GRIND 1 IRING 1

root@LAPTOP-T6LKEHA: /m  X + v

Start A New Game or Not? Y for Yes, N for Not:
```

三、 提高要求的实现方式

1、 tui 的实现方式

- 1) 使用 tui 库、crossterm 库。
- 2) 在需要输入时持续循环渲染一个输入框，一旦输入改变就重新渲染进入下一次循环，当用户按下 enter 时退出渲染（按键判断使用 crossterm 库）。
- 3) Tui 模式下将普通的字母表状态变成了键盘状态。
- 4) 通过输入框分割开了输入区和键盘（输出状态显示）区，输出位置通过 tui::terminal 的 set_cursor 函数决定，通过其 get_cursor 函数获取输出位置。
- 5) 在合适的时候（如输入完成之后）用 terminal.clear()清空屏幕，保证信息显示方式合理。
- 6) 在显示输出后用 sleep 函数为用户提供反应时间。
- 7) 注：由于调用的函数原因，此 tui 只能在合适的终端大小下正确显示，且不能在过程中拉伸终端。

2、 Wordle 求解（前 10 分）的实现方式

- 1) 新增命令行参数“-h”/“--hint”作为开启提示模式的标志。
- 2) 对于第一步猜测不给出推荐。
- 3) 从第二步猜测起，先从可用词列表筛选出所有可能的答案，方法与困难模式的操作

类似，新增加黄色字符不能在相同位置、红色字符的判断（如一个单词内只有红色的字符不能再出现、既有红色又有其他颜色的字符的出现个数需要变小等）。由于所有可能的答案可能较多（尤其是前两步），故此处暂时不输出所有可能的答案，只在下面计算得分后输出。

- 4) 虽然不可能为答案的单词可能提供更高的信息熵，但一方面为了更快给出推荐、一方面为了防止不可能为答案的词占据排名前几使下一轮不可能直接猜对，下一步推荐词的计算为在所有可能的答案的集合中计算类似信息熵的值，并按照计算值排序（计算值相同则按字典序小到大），给出得分最高的前十个单词。

四、 完成作业感想

在完成 wordle 游戏这个大作业的过程中，我感受到把一个游戏拆解为一步步可实现的任务，再从头开始实现整个游戏，在过程中进行合理的维护管理，是不容易的一件事。虽然很多游戏的开发流程不尽相同，但是如何合理地规划整个流程最终达到不错的游戏效果，同时让游戏易于维护仍是件难事。

而在写代码的过程中，我发现自学能力十分重要。各种第三方库的使用需要人认真阅读官方文档并转化为自己的知识而合理地运用到大作业中，这种过程和已有知识的运用截然不同。除此之外，与 OJ 题不同，对于游戏中用户的各种操作需要自行考虑所有可能的情况，设计多样的手工测试，对于不合法的操作也应做出合理的回复，这部分我还做得不够完美，需要继续体会测试设计的原则。

总之，这次大作业作为 rust 课程的第一个作业，让我有了多角度的收获。它一方面从语言角度让我体会了写 Rust 程序的不同风格，也从另一方面让我意识到程序维护、第三方库自学、设计测试等工作的重要性，会在未来的任务上给我带来启发。